

Unit-3

Feedforward Neural Networks

By: Arjun Saud, Asst. Prof. CDCSIT,TU

Perceptron

- The perceptron is the simplest form of a neural network used for the classifying linearly separable patterns. Patterns that lie on opposite sides of a hyperplane are called linearly separable patterns.
- Basically, perceptron consists of a single neuron with adjustable synaptic weights and bias. The algorithm used to adjust the free parameters of this neural network first appeared in a learning procedure developed by Rosenblatt (1958, 1962).

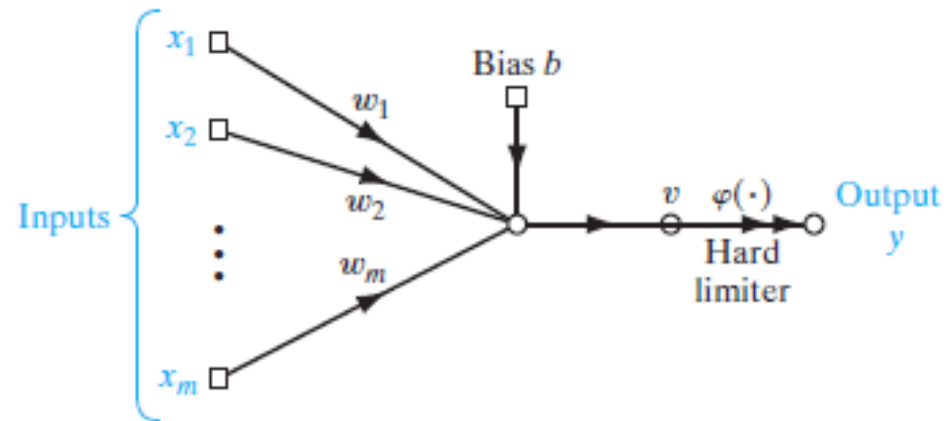
Perceptron

- Rosenblatt proved that if the patterns (vectors) used to train the perceptron are drawn from two linearly separable classes, then the perceptron algorithm converges and positions the decision surface in the form of a hyperplane between the two classes.
- The perceptron built around a *single neuron* is limited to performing pattern classification with only two classes. By expanding the output layer of the perceptron to include more than one neuron, we may correspondingly perform classification with more than two classes. However, the classes have to be linearly separable for the perceptron to work properly.

Perceptron

- Rosenblatt's perceptron is built around the *McCulloch–Pitts model* of a neuron.
- The summing node of the neural model computes a linear combination of the input. The resulting sum is applied to a hard limit activation function.
- The neuron produces an output equal to 1 if the hard limiter input is positive, and -1 if it is negative.

Perceptron



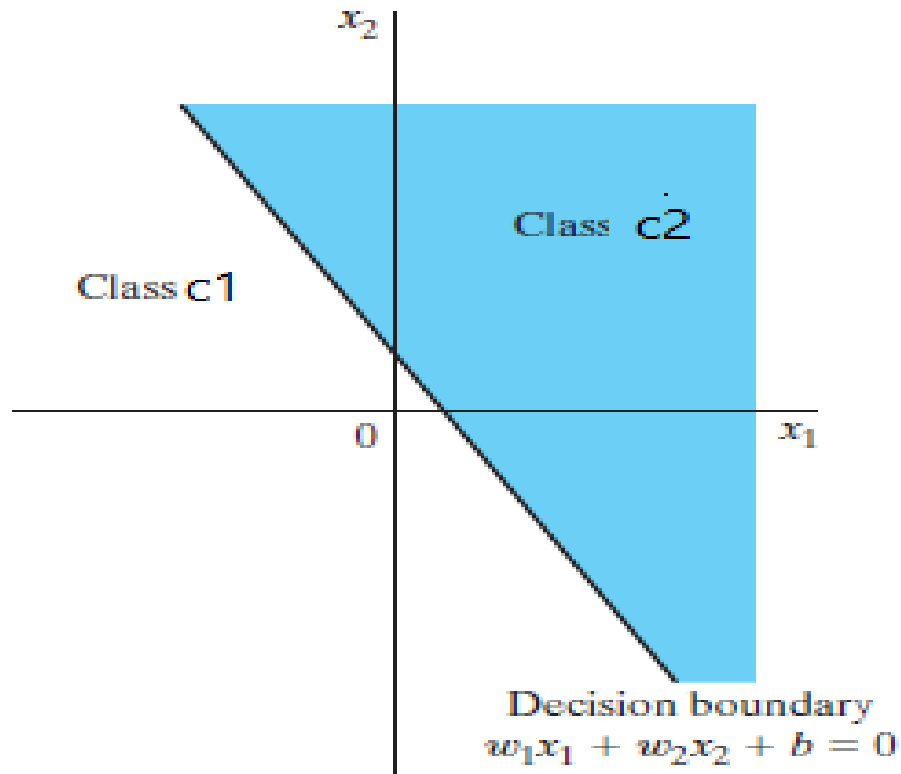
The hard limiter input (or induced local field) of the neuron is.

$$v = \sum_{i=1}^m w_i x_i + b$$

Perceptron

- The goal of the perceptron is to correctly classify the set of externally applied stimuli $x_1, x_2, \dots, x_m$ into one of two classes, c_1 or c_2 . The decision rule for the classification is to assign the point represented by the inputs $x_1, x_2, \dots, x_m$ to class c_1 if the perceptron output y is $+1$ and to class c_2 if it is -1 .
- In the simplest form of the perceptron, there are two decision regions separated by a *hyperplane*, which is defined by.

Perceptron



Perceptron

- Perceptron can be trained to show behavior of AND function and OR function because both are linearly separable. But, perceptron can not be trained to learn behavior of XOR function because it is not linearly separable.

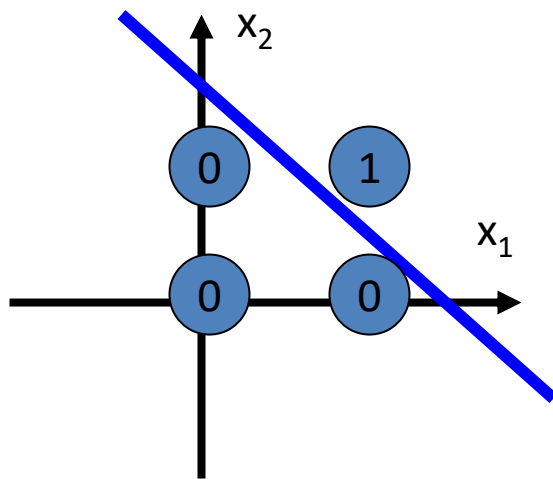


Fig: AND Function

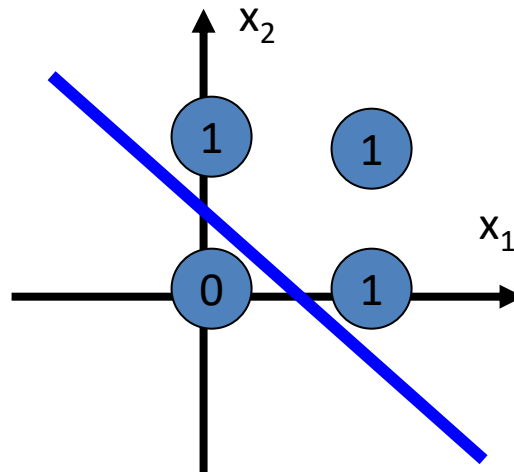


Fig: OR Function

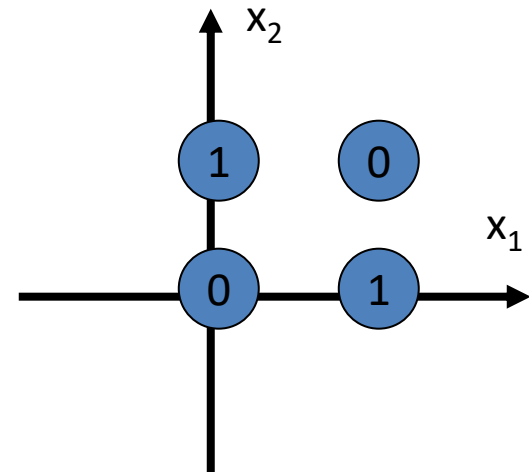


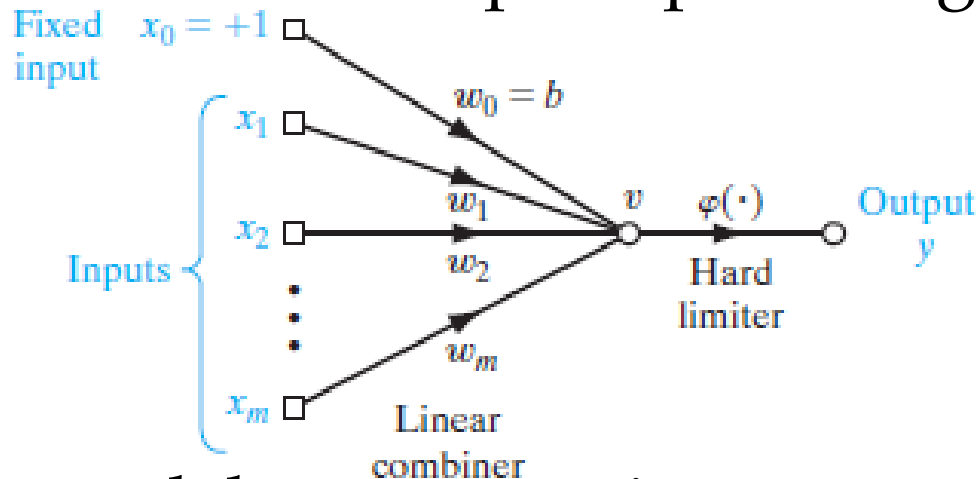
Fig: XOR Function

Perceptron Convergence Theorem

- The Perceptron convergence theorem states that *“for any data set which is linearly separable the Perceptron learning rule is guaranteed to find a solution in a finite number of steps, provided that learning rate is small.”*
- Perceptron learning rule is guaranteed to converge to a weight vector that correctly classifies the examples provided the training examples are linearly separable.
- Verification of this convergence theorem is provided in the example of perceptron learning rule.

Perceptron Learning Rule

- We can derive error correction rule for perceptron. Model of perceptron is given below,



- For this model, we can write

$$v = \sum_{i=0}^m w_i x_i$$

- Let $x = [x_0, x_1, x_2, \dots, x_m]$ and $w = [w_0, w_1, \dots, w_m]$

Perceptron Learning Rule

- Now, above equation can be written as:

$$v = w^T x$$

- Let us consider for n^{th} step input vector and weight vector is:

$$x = [x_0(n), x_1(n), \dots, x_m(n)] \text{ and } w = [w_0(n), w_1(n), \dots, w_m(n)]$$

- Now, for n^{th} step, above equation becomes

$$v = w(n)^T x(n)$$

- Suppose C_1 and C_2 are two classes for which perceptron needs to be trained. So, training set must include sufficient examples from both classes.

Perceptron Learning Rule

- Let H_1 and H_2 are set of training examples such that $H_1 \in C_1$ and $H_2 \in C_2$. Thus, Training Set(H) = $H_1 \cup H_2$.
- We know that perceptron puts data point in class C_1 if output is positive and in class C_2 if output is negative.
- This means, data point lies in class C_1 if $v > 0$ and in class C_2 if $v \leq 0$. Thus,
- For each $x \in H_1$, $w^T x > 0$ and
- For each $x \in H_2$, $w^T x \leq 0$

Perceptron Learning Rule

- Now, we can write weight update rule for n^{th} step as below.

Case I : Correct Classification

if $w^T(n)x(n) > 0$ and $x(n) \in C_1$ then $w(n+1) = w(n)$

if $w^T(n)x(n) \leq 0$ and $x(n) \in C_2$ then $w(n+1) = w(n)$

Case II : Missclassification

if $w^T(n)x(n) > 0$ and $x(n) \in C_2$ then $w(n+1) = w(n) - \alpha x(n)$

if $w^T(n)x(n) \leq 0$ and $x(n) \in C_1$ then $w(n+1) = w(n) + \alpha x(n)$

Perceptron Learning Rule

- The main theme of above weight change rule is:
 - Do not change weights in case of correct classification
 - Increase weights when perceptron output is smaller than actual target
 - Decrease weights when perceptron output is larger than actual target
- Thus, we can represent weight change rule using single equation as below:

$$w(n+1) = w(n) + \alpha x(n)(t(n) - y(n))$$

where, $y(n)$ is actual output, and $t(n)$ is target output

Perceptron Learning Rule

Perceptron Learning Algorithm

1. Initialize all weights and bias to zero
2. For each training vector s and target t perform steps 3 to 6
3. Set $x_i = s_i$ for $i = 1$ to n
4. Compute output using Hard limiter activation function as below

$$y_{in} = b + \sum_{i=1}^n w_i x_i \quad y = f(y_{in})$$

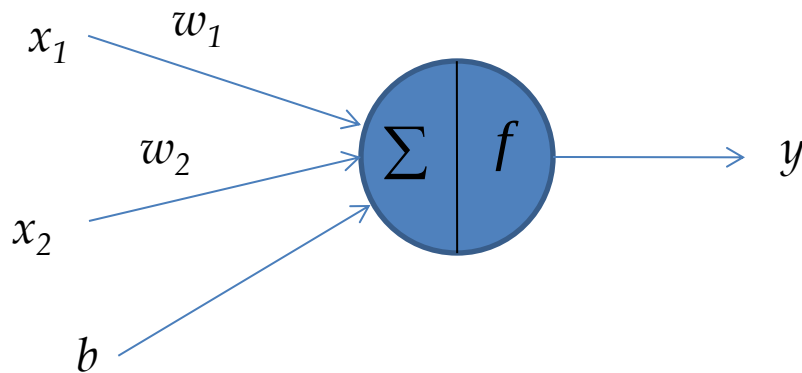
5. Adapt weights as: $w_i = w_i + \alpha(t - y)x_i$ for $i = 1$ to n
6. Adapt bias as: $b = b + \alpha(t - y)$
7. Test for Stopping Criteria

Perceptron Learning Rule

Perceptron Training

Example

Train the following perceptron by using given training set



x_1	x_2	t
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1

Perceptron Learning Rule

Solution: $\alpha=1$

Epoch #1

<i>Input</i>	w_1 (old)	w_2 (old)	b (old)	v	y	w_1 (new)	w_2 (new)	b (new)
(1,1,1)	0	0	0	0	0	$0+1*1*1=1$	$0+1*1*1=1$	$0+1*1=1$
(1,-1,-1)	1	1	1	1	1	$1+1*-2*1=-1$	$1+1*-2*-1=3$	$1+1*-2=-1$
(-1,1,-1)	-1	3	-1	3	1	$-1+1*-2*-1=1$	$3+1*-2*1=1$	$-1+1*-2=-3$
(-1,-1,-1)	1	1	-3	-5	-1	$1+1*0*-1=1$	$1+1*0*-1=1$	$-3+1*0=-3$

$$\varphi(x) = \begin{cases} 1 & \text{if } x > 0 \\ -1 & \text{if } x < 0 \\ 0 & \text{if } x = 0 \end{cases}$$

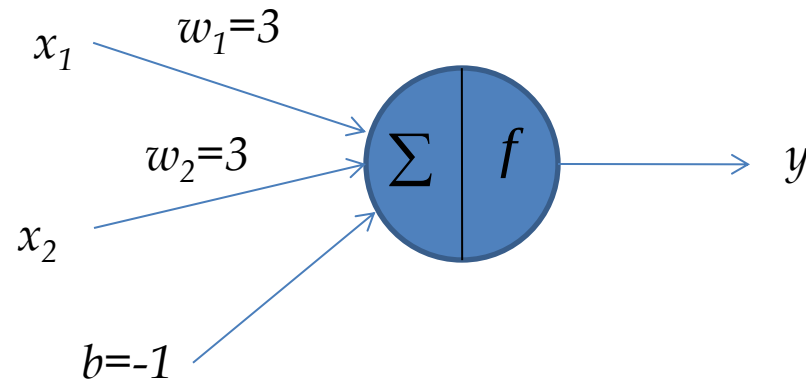
Assimed hard Limiter Activation function

Perceptron Learning Rule

Epoch #2

<i>Input</i>	w_1 (old)	w_2 (old)	b (old)	v	y	w_1 (new)	w_2 (new)	b (new)
(1,1,1)	1	1	-3	-1	-1	$1+1*2*1=3$	$1+1*2*1=3$	$-3+1*2=-1$
(1,-1,-1)	3	3	-1	-1	-1	$3+1*0*1=3$	$3+1*0*-1=3$	$-1+1*0=-1$
(-1,1,-1)	3	3	-1	-1	-1	$2+1*0*-1=3$	$3+1*0*1=3$	$-1+1*0=-1$
(-1,-1,-1)	3	3	-1	-7	-1	$3+1*0*-1=3$	$3+1*0*-1=3$	$-1+1*0=-1$

Thus, Final Neuron is



Perceptron Learning Rule

Example 2

Train perceptron using given training set and predict class for the input (6,82) and (5.3,52)

<i>Height(x_1)</i>	<i>Weight(x_2)</i>	<i>Class(t)</i>
5.9	75	Male
5.8	86	Male
5.2	50	Female
5.4	55	Female
6.1	85	Male
5.5	62	Female

Perceptron Learning Rule

Solution

Normalize the input and encode target

Use min-max scaling $y = (x - \min) / (\max - \min)$

Scaling of 5.9 :- $(5.9 - 5.2) / (6.1 - 5.2) = 0.7 / 0.9 = 0.778$

Similarly normalize other values

Height(x_1)	Weight(x_2)	Class(t)
5.9	75	Male
5.8	86	Male
5.2	50	Female
5.4	55	Female
6.1	85	Male
5.5	62	Female



Height(x_1)	Weight(x_2)	Class(t)
0.778	0.694	1
0.667	1.000	1
0.000	0.000	-1
0.222	0.139	-1
1.000	0.972	1
0.333	0.333	-1

Perceptron Learning Rule

Solution: $\alpha=1$

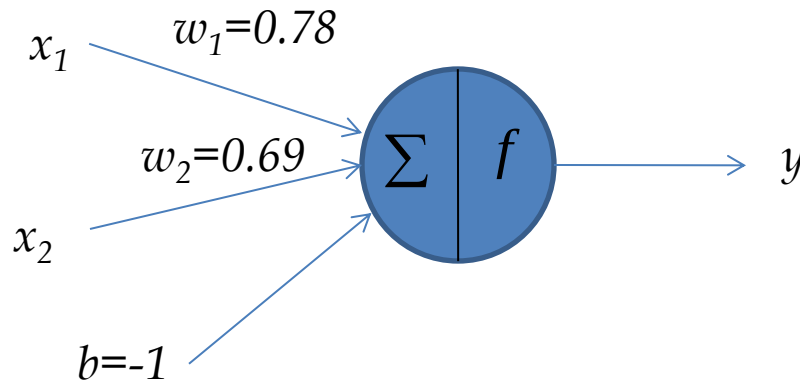
Epoch #1

<i>Input</i>	w_1 (old)	w_2 (old)	b (old)	v	y	w_1 (new)	w_2 (new)	b (new)
0.78,0.69,1	0	0	0	0	0	$0+1*1*0.78=0.78$	$0+1*1*0.69=0.69$	$0+1*1=1$
0.67,1,1	0.78	0.69	1	2.21	1	0.78	0.69	1
0,0,-1	0.78	0.69	1	1	1	$0.78+1*-2*0=0.78$	$0.69+1*-2*0=0.69$	$1+1*-2=-1$
0.22,0.14,-1	0.78	0.69	-1	-0.73	-1	0.78	0.69	-1
1,0.97,1	0.78	0.69	-1	0.45	1	0.78	0.69	-1
0.33,0.33,1-	0.78	0.69	-1	-0.51	-1	0.78	0.69	-1

Perceptron Learning Rule

Solution Contd..

Thus, Final perceptron is as below:



For (6,82)

Normalized value of 6 = $(6-5.2)/(6.1-5.2)=0.8/0.9=0.89$

Normalized value of 82 = $(82-50)/(86-50)=32/36=0.89$

Perceptron Learning Rule

Solution Contd..

$$v = w_1x_1 + w_2x_2 + b = 0.78 * 0.89 + 0.69 * 0.89 - 1 = 0.31$$

Thus, $y=1$

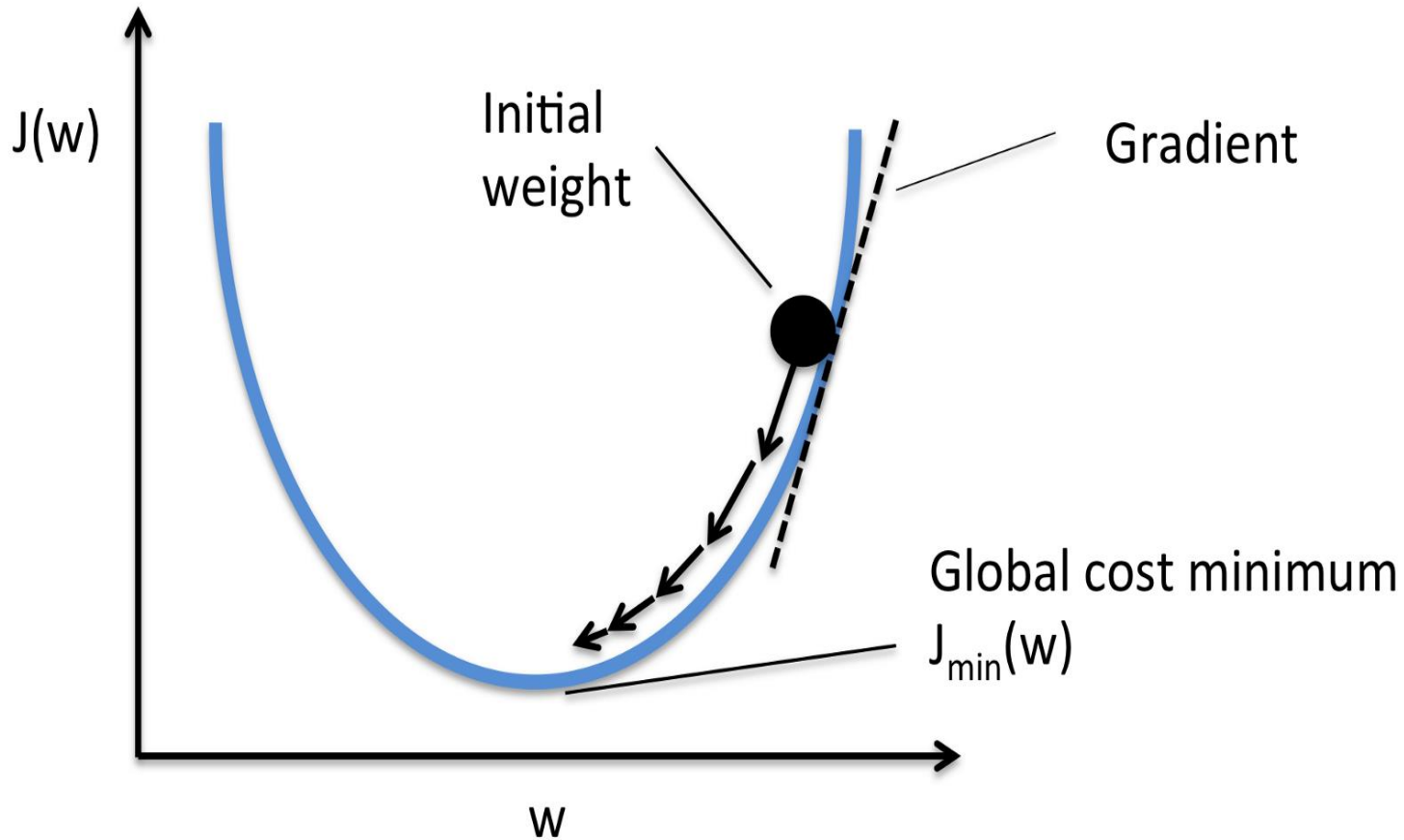
Hence, Predicted class for (6,82) is Male.

Similarly, we can predict class for (5.3,52)

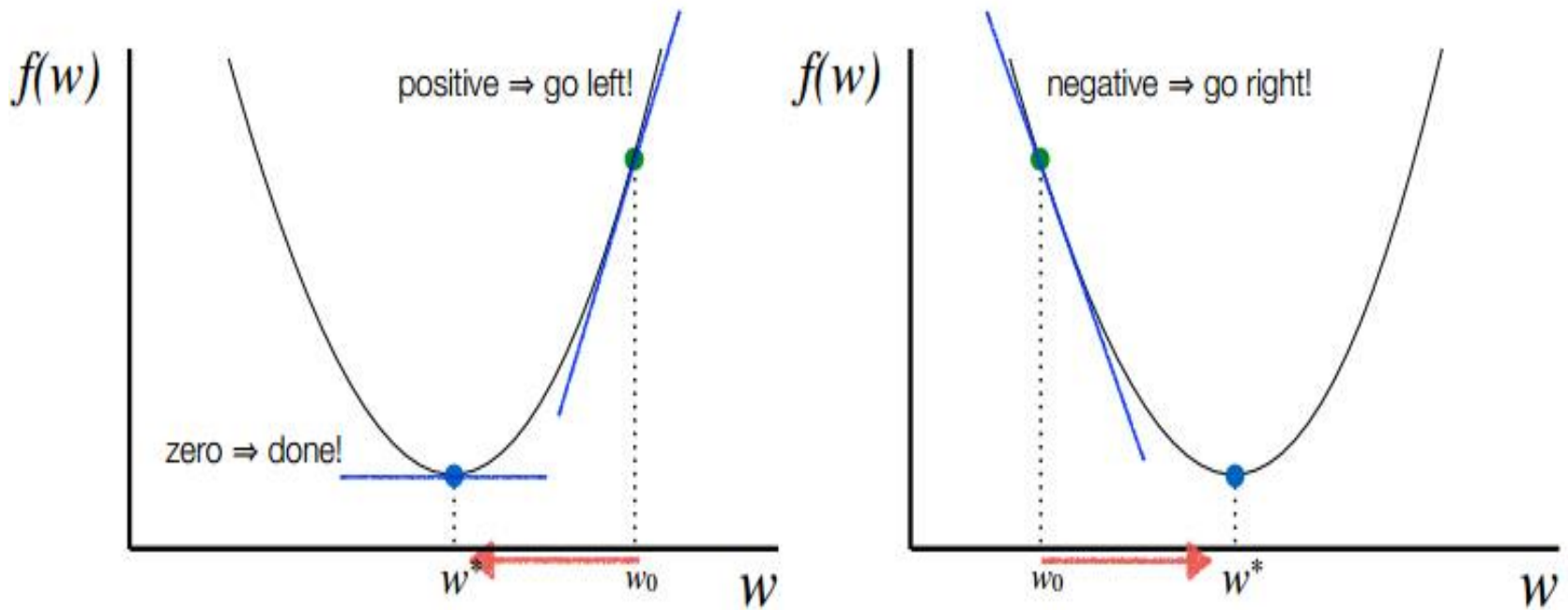
Gradient Descent

- Gradient descent is an optimization algorithm used to minimize some convex function by iteratively moving in the direction of steepest descent as defined by the negative of the gradient.
- In machine learning, we use gradient descent to update the parameters of our model. Parameters refer to coefficients in Logistic Regression and weights in neural networks.

Gradient Descent



Gradient Descent



Gradient Descent

- If f is function to be minimized (cost function), Gradient descent changes the parameters of learning model iteratively as below:

$$w = w - \alpha \frac{df}{dw}$$

where, α is learning rate and w is parameter to be optimized

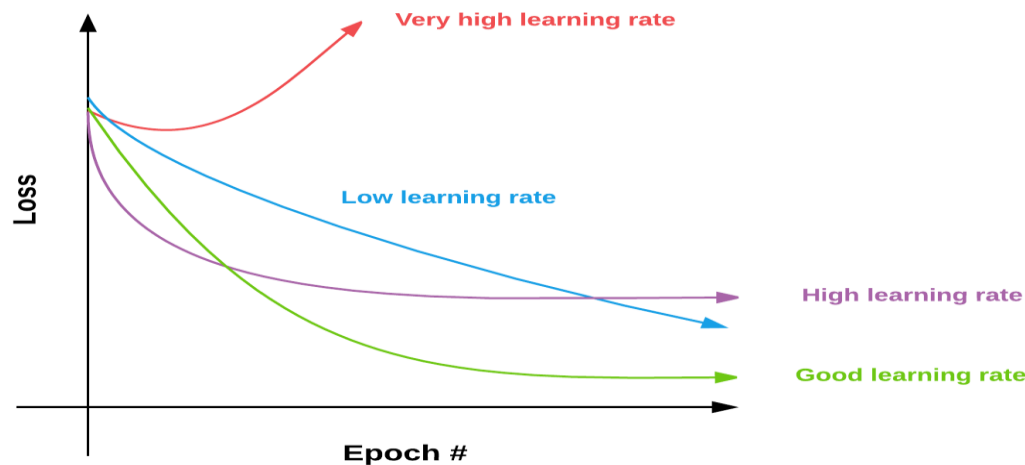
- GD simply measures the change in all weights with regard to the change in error. We can also think of a gradient as the slope of a function.

Gradient Descent

- The higher the gradient, the steeper the slope and the faster a model can learn. But if the slope is zero, the model stops learning. Said it more mathematically, a gradient is a partial derivative with respect to its inputs.
- How big the steps are that Gradient Descent takes into the direction of the local minimum are determined by the learning rate.

Gradient Descent

- With a high learning rate we can cover more ground each step, but we risk overshooting the lowest point. A low learning rate is more precise, but calculating the gradient is time-consuming, so it will take us a very long time to get to the bottom.



Batch Perceptron Algorithm

- When training set is large, taking each training example one by one and updating perceptron weights is time consuming tasks.
- In such case batch perceptron algorithm is suitable. The main concept behind batch perceptron algorithm is: “Process training examples in batches. Size of batch should be defined on the basis of size of training set. Commonly used batch size are 16, 32, 64,128, etc. ”
- Then, update weights for $(n+1)^{\text{th}}$ step by taking sum of all misclassifications in the batch of training example.

Batch Perceptron Algorithm

- For the training examples that are incorrectly classified by the algorithm:

$$w^T x t < 0$$

- Therefore, to minimize cost function for Perceptron, we can write:

$$J(w) = \sum_{x_i \in m} -w^T x_i t_i$$

where m is the set of misclassified training examples

Batch Perceptron Algorithm

- Differentiate above cost function w.r.t. w to get gradient vector.

$$\nabla J(w) = \sum_{x_i \in m} -x_i t_i$$

- In the *steepest descent method*, the adjustment to the weight vector w **at each time step** of the algorithm is applied in a direction *opposite to the gradient vector*.
- Thus, weight update rule for batch perceptron algorithm can be written as

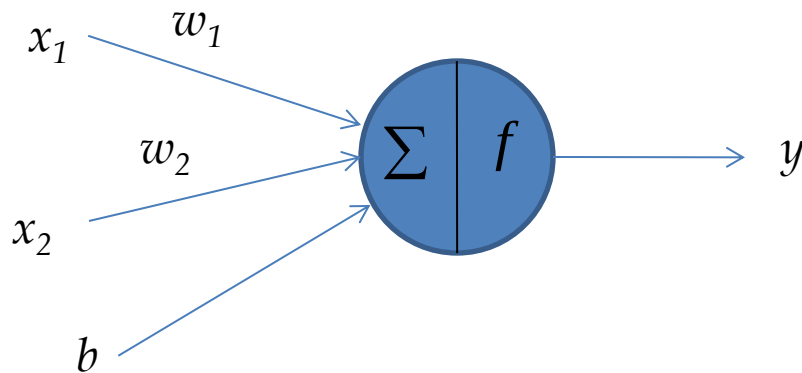
$$w(n+1) = w(n) - \alpha \nabla J(w) = w(n) + \alpha \sum_{x_i \in m} x_i t_i$$

Batch Perceptron Algorithm

Perceptron Training

Example

Train the following perceptron using batch perceptron algorithm.



x_1	x_2	t
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1

Multi-Layer Feedforward NN

- A multilayer feed-forward network consists of an input layer, one or more hidden layers, and an output layer. Computations take place in the hidden and output layers only.
- The input signal propagates through the network in a forward direction layer-by-layer. Such neural networks are called multilayer perceptrons (MLPs).
- They have been successfully applied to many difficult and diverse problems.
- Multilayer perceptrons are typically trained using so-called error backpropagation algorithm. This is a supervised error-correction learning algorithm.

Multi-Layer Feedforward NN

Properties of MLPs

- Each neuron has a smooth (differentiable everywhere) nonlinear activation function. This is usually a sigmoidal nonlinearity defined by the logistic function

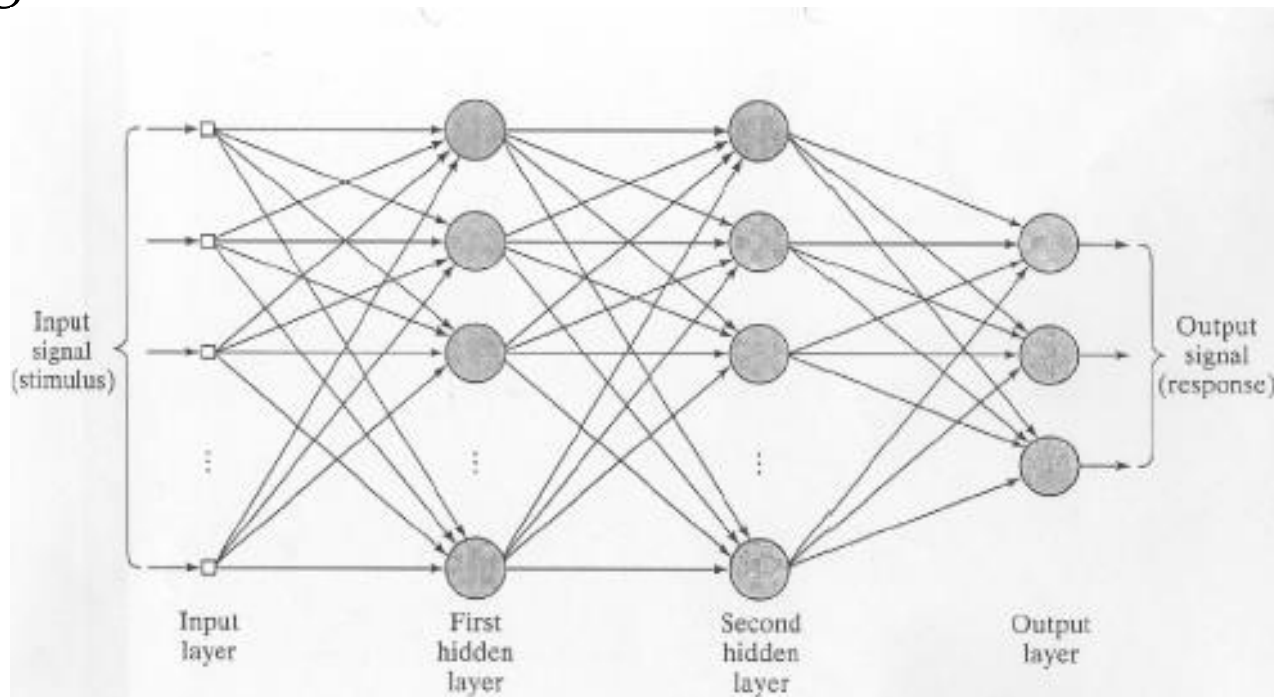
$$y_j = \frac{1}{1 + \exp(-v_j)}, \text{ where } v_j \text{ is weighted sum of inputs}$$

Note: Nonlinearities are important: otherwise the network could be reduced to a linear single-layer perceptron.

- The network contains hidden layer(s), enabling learning complicated tasks and mappings.
- The network has a high connectivity.

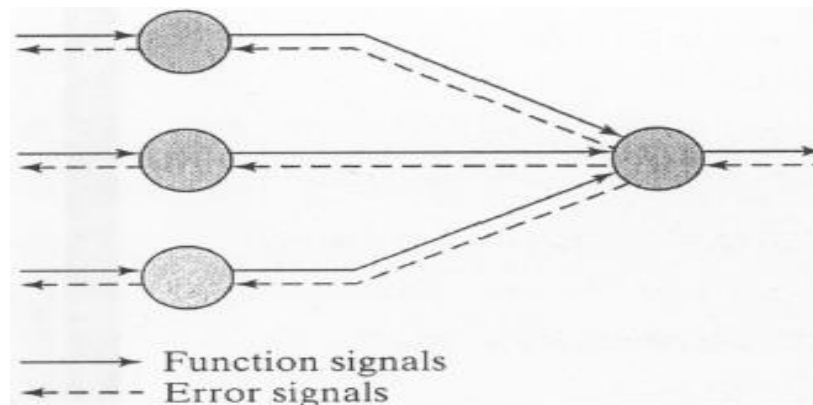
Multi-Layer Feedforward NN

- An architectural graph of a fully connected multilayer perceptron with two hidden layers and an output layer is given below.



Multi-Layer Feedforward NN

- Two kinds of signals appear in the MLP networks: Function Signal and Error Signal.
 - **Function Signals:** Function signals are the Input signals that propagates in forward direction through the network and produces output signals in the last phase.
 - **Error Signals:** These are the signals originate at output neurons, and propagate layer by layer in backward direction through the network.



Multi-Layer Feedforward NN

- Each hidden or output neuron performs two computations:
 1. The computation of the function signal appearing at its output. This is a nonlinear function of the input signal and synaptic weights of that neuron.
 2. The computation of an estimate of the gradient vector, needed in the backward pass.

Batch and Online Learning

Batch Learning

- In the batch method of supervised learning, adjustments to the synaptic weights of the multilayer perceptron are updated *after the presentation of all the N examples in the training sample T that constitute one *epoch of training*.*
- *In other words, the cost function for batch learning is defined by the average error energy and adjustments to the synaptic weights of the multilayer perceptron are made on an *epoch-by-epoch basis*.*

Batch and Online Learning

Batch Learning

- With the method of gradient descent used to perform the training, the advantages of batch learning include the following:
 - *Accurate estimation of the gradient vector, **thereby guaranteeing**, convergence of the method of steepest descent to a local minima for non-convex surfaces and at global minima for convex surface;*
 - *parallelization of the learning process.*

Batch and Online Learning

Batch Learning

- For large datasets, batch learning is computationally intractable because it is not possible to store entire dataset into memory at once.
- Since it updates weights less frequently, it is computationally faster.

Batch and Online Learning

Online Learning

- In the on-line method of supervised learning, adjustments to the synaptic weights of the multilayer perceptron are performed on an example-by-example basis. The cost function to be minimized is therefore the total instantaneous error energy for one training example.
- Due to noise gradient descent calculated for each of the training examples, online learning jumps here and there which makes it difficult to converge in global minima. Due to this behavior of online learning, it is also called stochastic method.

Batch and Online Learning

Online Learning

- Besides this, parallelization of online learning method is not possible.
- Since it updates weights example by example basis, it is computationally slower than batch learning. But, it is computationally tractable for the large training set because we do not need to store large dataset into memory at once.

Batch and Online Learning

Trade-off Between Batch and Online Learning

- If we look above discussion, we can conclude that Batch learning is better approach. But, it is computationally intractable for large datasets.
- On the other hand, online learning is computationally tractable for large datasets but it is difficult to converge in global minim.
- Thus, the better approach is to use mini-batch learning approach, where entire dataset is divided into batches of size 16 or 32 or 64 or 128 etc., on the basis of need and synaptic weights of the multilayer perceptron are updated *after the presentation of all the N examples in a mini-batch.*

Backpropagation Algorithm

- A popular method for the training of multilayer perceptrons is the back-propagation algorithm. The training proceeds in two phases:
 - **Forward Phase:** In this phase, the synaptic weights of the network are fixed and the input signal is propagated through the network, layer by layer, until it reaches the output. Thus, in this phase, changes are confined to the activation potentials and outputs of the neurons in the network.

Backpropagation Algorithm

- **Backward Phase:** In this phase, an error signal is produced by comparing the output of the network with a desired response. The resulting error signal is propagated in backward direction through the network, again layer by layer. In this second phase, successive adjustments are made to the synaptic weights of the network

Backpropagation Algorithm

- The error signal at the output of neuron j at iteration n is given by
$$e_j(n) = d_j(n) - y_j(n)$$
where $d_j(n)$ is desired output and $y_j(n)$ is output produced by MLP
- The total instantaneous error energy $E(n)$ for all the neurons in the output layer is therefore

$$E(n) = \frac{1}{2} \sum_{j \in C} e_j^2(n)$$

where C is the set of neurons in output layer

Backpropagation Algorithm

- Let N be the total number of training vectors (examples). Then the average squared error is:

$$E_{avg} = \frac{1}{N} \sum_{n=1}^N E(n)$$

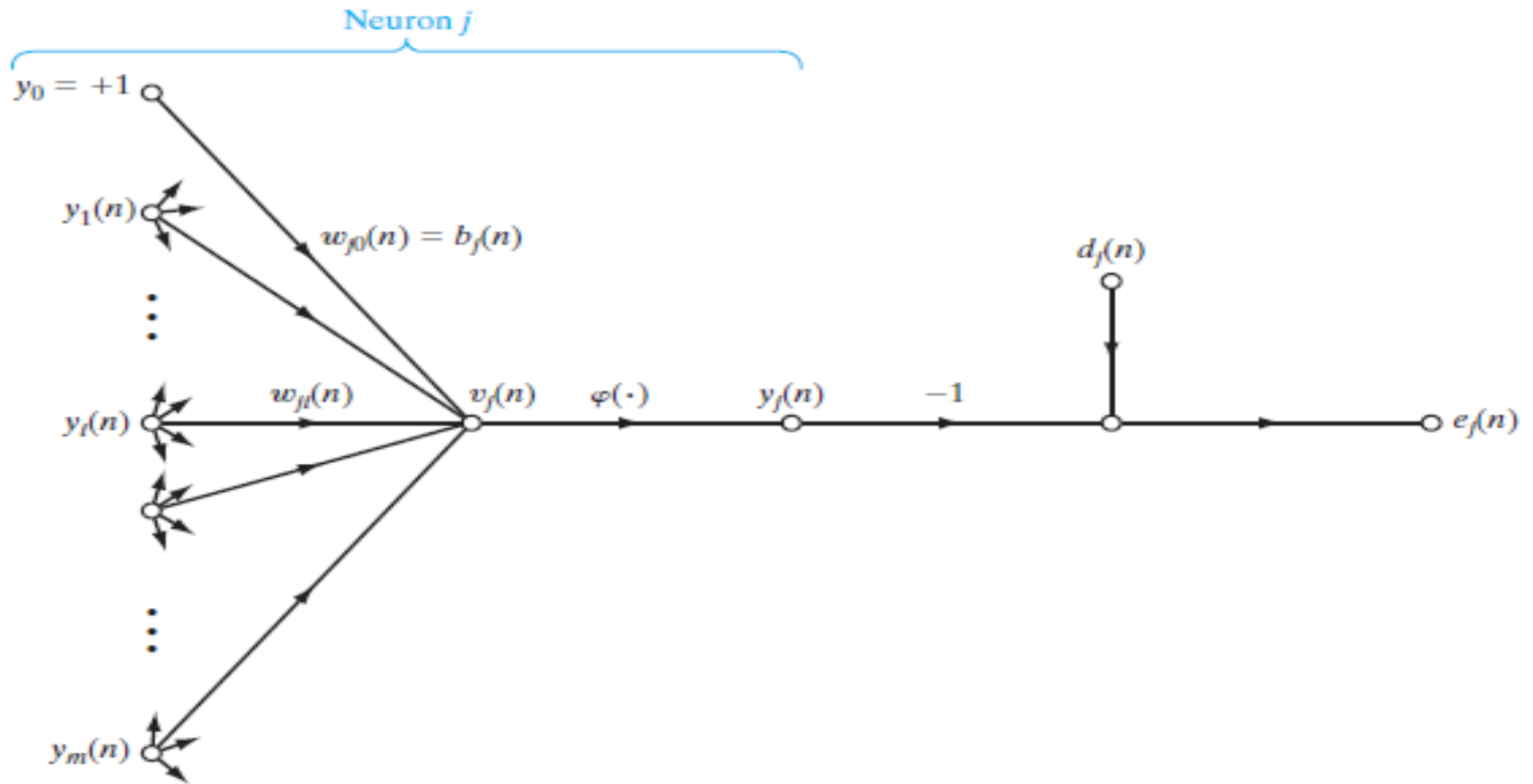
- Consider the neuron j as given in next slide. The local field $v_j(n)$ and output $y_j(n)$ of neuron j is given by:

$$v_j = \sum_{i=0}^m w_{ji}(n) y_i(n)$$

$$y_j(n) = \varphi_j(v_j(n))$$

where y_i is output of neuron i and w_{ji} is weight of link from neuron i to j

Backpropagation Algorithm



Backpropagation Algorithm

- The correction $\Delta w_{ji}(n)$ made to the weight is proportional to the partial derivative $\partial E(n)/\partial w_{ji}(n)$ of instantaneous error
- Using the chain rule of calculus, this gradient can be expressed as follows:

$$\frac{\partial E(n)}{\partial w_{ji}(n)} = \frac{\partial E(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)} \quad (1)$$

- We can get following partial derivatives

$$\frac{\partial E(n)}{\partial e_j(n)} = e_j(n) \quad \frac{\partial e_j(n)}{\partial y_j(n)} = -1 \quad \frac{\partial y_j(n)}{\partial v_j(n)} = \phi'_j(v_j(n))$$

Backpropagation Algorithm

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n)$$

- Putting all partial derivatives in equation 1, we get

$$\frac{\partial E(n)}{\partial w_{ji}(n)} = -e_j(n) \phi'_j(v_j(n)) y_i(n) \quad (2)$$

- The correction applied to the weight is defined by

$$\Delta w_{ji} = -\alpha \frac{\partial E(n)}{\partial w_{ji}(n)} \quad (3)$$

Backpropagation Algorithm

- Using equation (2), equation (3) can be written as

$$\Delta w_{ji} = \alpha e_j(n) \phi_j'(v_j(n)) y_i(n) \quad (4)$$

- This can be written as

$$\Delta w_{ji} = \alpha \delta_j(n) y_i(n)$$

$$\text{where } \delta_j(n) = e_j(n) \phi_j'(v_j(n)) \quad (5)$$

- The error term in equation 4 and 5 depends upon location of neuron in the MLP. There can be two possible scenarios.

Backpropagation Algorithm

Case I: Neuron j is output layer neuron

- The desired response $d_j(n)$ for the neuron j is directly available. Computation of the error $e_j(n)$ is straightforward in this case. We can use equation 4 or 5 to calculate weight update term.

Case II: Neuron j is hidden layer neuron

- There is no desired response available for neuron j. The error signal for a hidden neuron must be determined recursively in terms of the error signals of all neurons connected to it as below:

$$\delta_j(n) = \varphi_j'(v_j(n)) \sum_k \delta_k(n) w_{kj}(n) \quad (6)$$

Backpropagation Algorithm

Algorithm

1. Initialize all weights and biases in *network*
2. While terminating condition is not satisfied
3. for each training tuple X in D
4. for each input layer unit j : $y_j = x_j$; // output of an input unit is its actual input value
5. for each hidden or output layer unit j

compute the net input of unit j with respect to the previous layer,

$$v_j = \sum_i w_{ji} y_i$$

compute the output of each unit j

$$y_j = \frac{1}{1 + e^{-v_j}}$$

Learning in ANN

Algorithm

6. for each unit j in the output layer

$$\delta_j = e_j \varphi_j'(v_j) = \varphi_j'(v_j)(d_j - y_j)$$

7. for each unit j in the hidden layers, from the last to the first hidden layer

$$\delta_j = \varphi_j'(v_j) \sum_k \delta_k w_{kj}$$

8. for each weight w_{ji} in *network*

$$\Delta w_{ji} = \alpha \delta_j y_i$$

$$w_{ji} = w_{ji} + \Delta w_{ji} \quad // \text{Weight Update}$$

Backpropagation Algorithm

- Backpropagation algorithm needs to compute $\phi'_j(v_j)$ to compute δ_j . This depends upon choice of activation function.

Case I: Logistic Activation Function

$$\phi(x) = y = \frac{1}{1 + e^{-x}} \Rightarrow \phi'(x) = y(1 - y)$$

If we consider general sigmoid function, then

$$\phi(x) = y = \frac{1}{1 + e^{-ax}} \Rightarrow \phi'(x) = ay(1 - y)$$

Note: Compute Derivatives of Logistic activation function.

Backpropagation Algorithm

Case II: Hyperbolic Tangent Function

$$\varphi(x) = y = \frac{e^x - e^{-x}}{e^x + e^{-x}} \Rightarrow \varphi'(x) = (1+y)(1-y)$$

If we consider general Tanh function, then

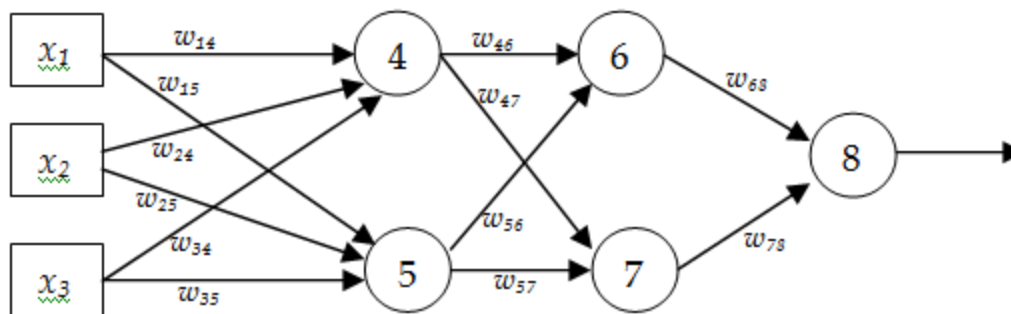
$$\varphi(x) = y = a \tanh(bx) \Rightarrow \varphi'(x) = \frac{b}{a} (a+y)(a-y)$$

Note: Compute Derivatives of Hyperbolic Function.

Backpropagation Algorithm

Example

- Consider a MLP given below. Let the learning rate be 1. The initial weights of the network are given in the table below. Assume that first training tuple is (1, 0, 1) and its target output is 1. Calculate weight updates by using back-propagation algorithm. Assume $\varphi(x) = \frac{1}{1 + e^{-x}}$.



w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{47}	w_{56}	w_{57}	w_{68}	w_{78}
0.6	0.4	0.2	-0.3	0.7	-0.6	0.4	0.7	0.1	0.8	0.2	0.5

Backpropagation Algorithm

Solution

Forward Pass

$$v_4 = 1 * 0.6 + 0 * 0.2 + 1 * 0.7 = 1.3$$

$$y_4 = 1 / (1 + e^{-1.3}) = 0.786$$

$$v_5 = 1 * 0.4 + 0 * (-0.3) + 1 * (-0.6) = -0.2$$

$$y_5 = 1 / (1 + e^{0.2}) = 0.45$$

$$v_6 = 0.786 * 0.4 + 0.45 * 0.1 = 0.36$$

$$y_6 = 1 / (1 + e^{-0.36}) = 0.59$$

$$v_7 = 0.786 * 0.7 + 0.45 * 0.8 = 0.91$$

$$y_7 = 1 / (1 + e^{-0.91}) = 0.71$$

$$v_8 = 0.59 * 0.2 + 0.71 * 0.5 = 0.47$$

$$y_8 = 1 / (1 + e^{-0.47}) = 0.61$$

Backpropagation Algorithm

Solution

Backward Pass

$$\delta_8 = y_8(1 - y_8)(d_8 - y_8) = 0.61 * (1 - 0.61) * (1 - 0.61) = 0.093$$

$$\delta_7 = y_7(1 - y_7)\delta_8 w_{87} = 0.71 * (1 - 0.71) * 0.093 * 0.5 = 0.0096$$

$$\delta_6 = y_6(1 - y_6)\delta_8 w_{86} = 0.59 * (1 - 0.59) * 0.093 * 0.2 = 0.0045$$

$$\delta_5 = y_5(1 - y_5)(\delta_7 w_{75} + \delta_6 w_{65})$$

$$= 0.45 * (1 - 0.45) * (0.0096 * 0.8 + 0.0045 * 0.1) = 0.002$$

$$\delta_4 = y_4(1 - y_4)(\delta_7 w_{74} + \delta_6 w_{64})$$

$$= 0.786 * (1 - 0.786) * (0.0096 * 0.7 + 0.0045 * 0.4) = 0.0014$$

Backpropagation Algorithm

Solution

Update Weights

$$w_{14} = w_{14} + 1 * \delta_4 * y_1 = 0.6 + 1 * 0.0014 * 1 = ?$$

$$w_{15} = w_{15} + 1 * \delta_5 * y_1 = ?$$

$$w_{24} = w_{24} + 1 * \delta_4 * y_2 = ?$$

$$w_{25} = w_{25} + 1 * \delta_5 * y_2 = ?$$

$$w_{34} = w_{34} + 1 * \delta_4 * y_3 = ?$$

$$w_{35} = w_{35} + 1 * \delta_5 * y_3 = ?$$

$$w_{46} = w_{46} + 1 * \delta_6 * y_4 = ?$$

$$w_{47} = w_{47} + 1 * \delta_7 * y_4 = ?$$

$$w_{56} = w_{56} + 1 * \delta_6 * y_5 = ?$$

$$w_{57} = w_{57} + 1 * \delta_7 * y_5 = ?$$

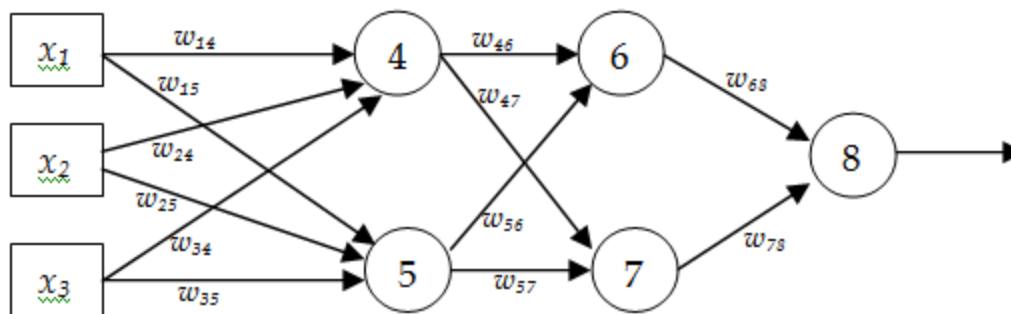
$$w_{68} = w_{68} + 1 * \delta_8 * y_6 = ?$$

$$w_{78} = w_{78} + 1 * \delta_8 * y_7 = ?$$

Backpropagation Algorithm

HW

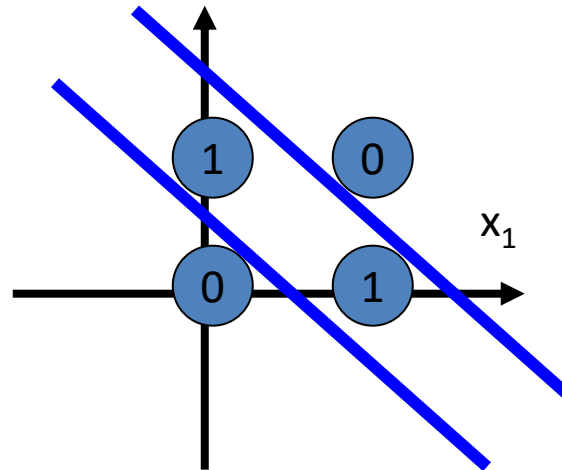
- Consider a MLP given below. Let the learning rate be 1. The initial weights of the network are given in the table below. Assume that first training tuple is (1, 0, 1) and its target output is 1. Calculate weight updates by using back-propagation algorithm. Assume $\varphi(x) = y = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{47}	w_{56}	w_{57}	w_{68}	w_{78}
0.6	0.4	0.2	-0.3	0.7	-0.6	0.4	0.7	0.1	0.8	0.2	0.5

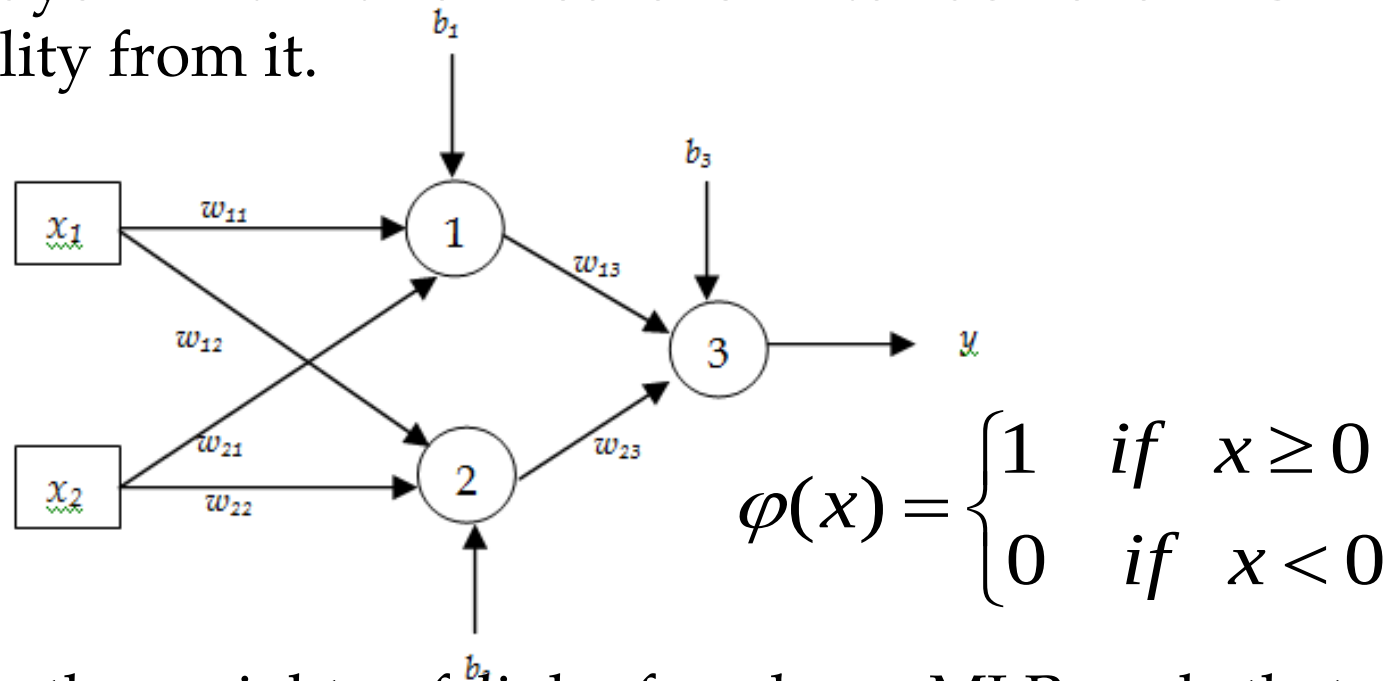
XOR Problem

- We know that single layer perceptron can be trained to classify patterns that is linearly separable.
- However, XOR function is non-linearly separable as shown in the figure given below. Thus, we can not train single layer perceptron for XOR function. This, problem is called XOR Problem.



XOR Problem

- We need to use multilayer perceptron that contains one hidden layer with two neurons to achieve XOR functionality from it.



- Determine the weights of links for above MLP such that MLP functions as XOR. Assume that threshold activation function (as shown above) is used in each of the neuron.

XOR Problem

- We can have following realization of AND, OR, and NOR gates using perceptron.

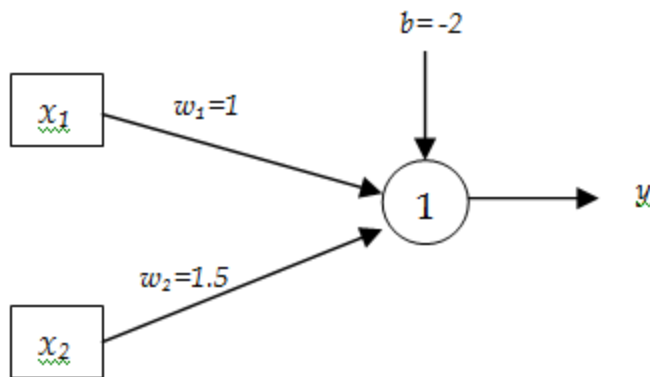


Fig: AND Function

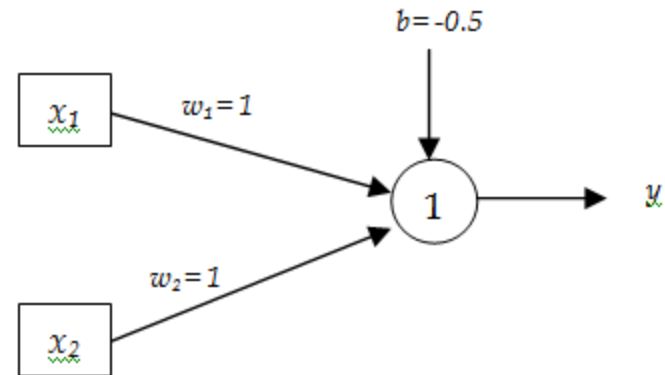


Fig: OR Function

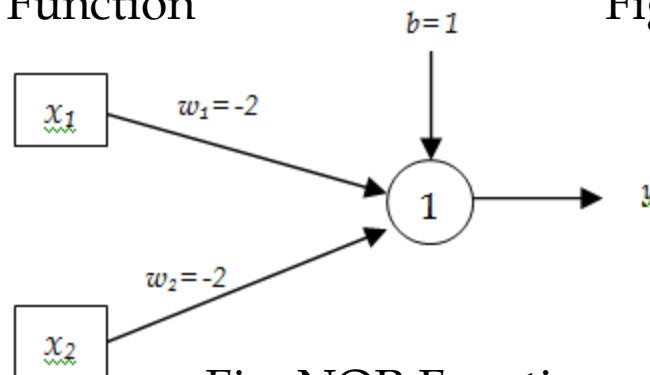


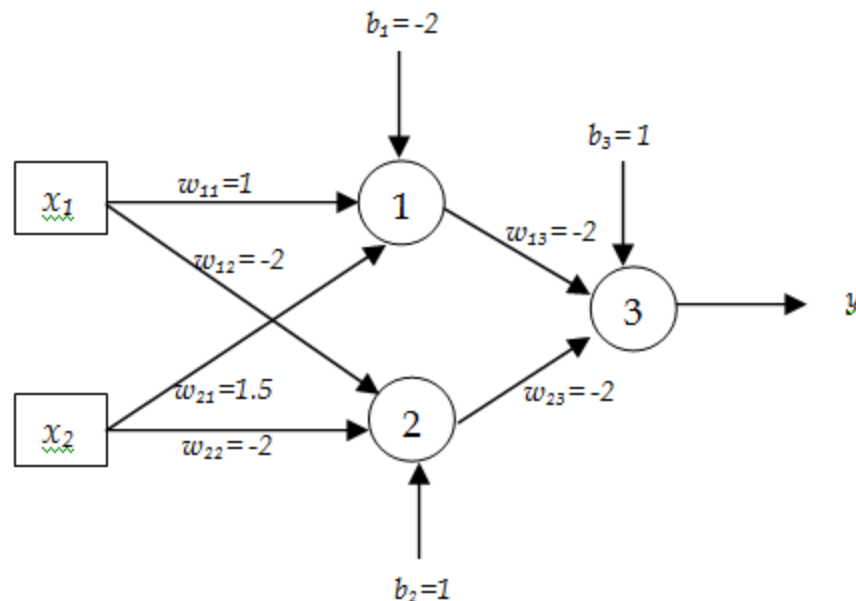
Fig: NOR Function

XOR Problem

- XOR function can be represented in terms of function of AND and NOR functions as below.

$$\text{NOR}(\text{AND}(x_1, x_2), \text{NOR}(x_1, x_2))$$

- Thus, Weights of MLP that acts as XOR function can be set as below



XOR Problem

CW

- Realize NOT and NAND function using perceptron.
- Can we realize XNOR function using perceptron? If yes, realize it using perceptron. Otherwise, realize XNOR function using MLP.

Jacobian and Hessian

- The Jacobian is a matrix of all the first-order partial derivatives of a vector-valued function. In the neural network case, it is an N -by- W matrix, where N is the number of entries in our training set and W is the total number of parameters (weights) of our network.
- It can be generated by taking the partial derivatives of each output with respect to each weight.
- Each row of the Jacobian corresponds to a particular example in the training sample.

Jacobian and Hessian

$$J = \begin{bmatrix} \frac{\partial F(x_1, w)}{\partial w_1} & \dots & \frac{\partial F(x_1, w)}{\partial w_W} \\ \vdots & \ddots & \vdots \\ \frac{\partial F(x_N, w)}{\partial w_1} & \dots & \frac{\partial F(x_N, w)}{\partial w_W} \end{bmatrix}.$$

- While it is a good exercise to compute the gradient of a neural network with respect to a single parameter (e.g., a single weight), in practice this tends to be quite slow. This approach can't take advantage of vectorization. Thus, for vectorized implementation of backpropagation algorithm, it is necessary to use Jacobian.

Jacobian and Hessian

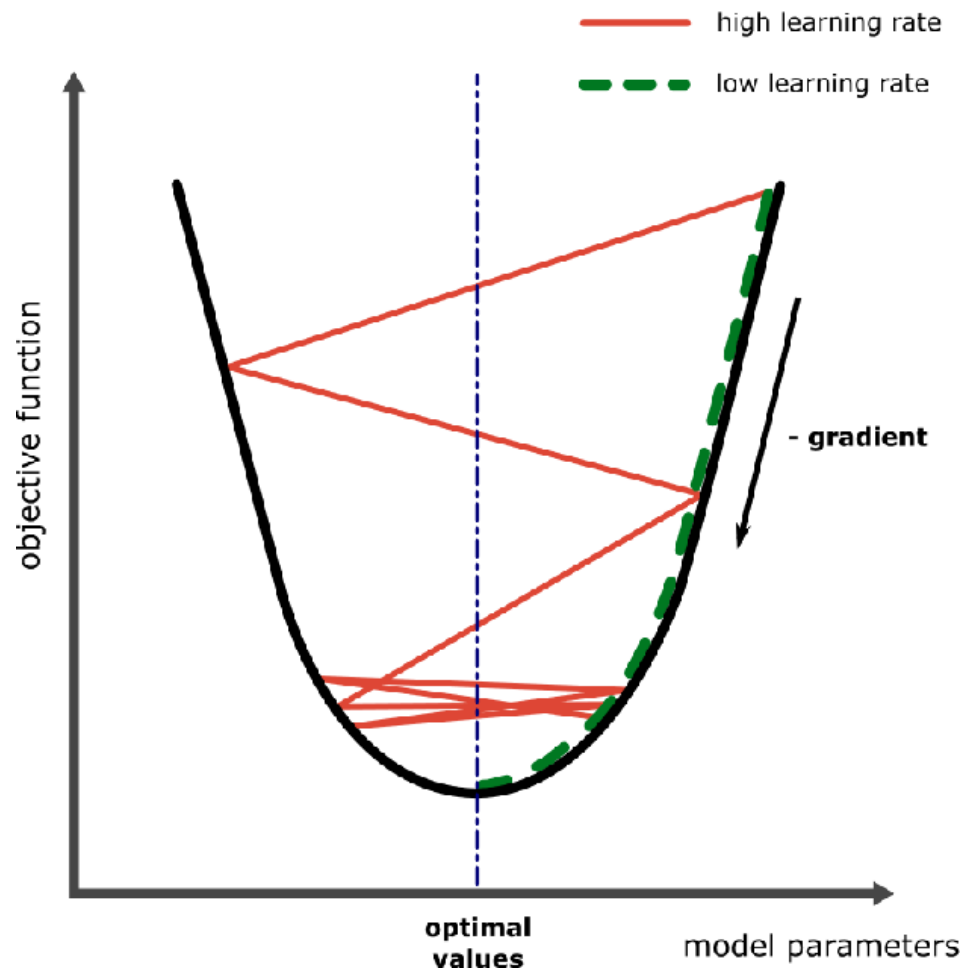
- Hessian of a cost function is square matrix whose elements are second order partial derivative of the cost function with respect to each weight in the MLP.

$$H = \frac{\partial^2 f}{\partial w^2} = \begin{bmatrix} \frac{\partial^2 f}{\partial w_1^2} & \frac{\partial^2 f}{\partial w_1 w_2} & \dots & \frac{\partial^2 f}{\partial w_1 w_n} \\ \vdots & \vdots & & \vdots \\ \frac{\partial^2 f}{\partial w_n w_1} & \frac{\partial^2 f}{\partial w_n w_2} & \dots & \frac{\partial^2 f}{\partial w_n^2} \end{bmatrix}$$

Learning Rate Annealing

- One of the key hyper parameters to set in order to train a neural network is the *learning rate*.
- If learning rate is set too low, training will progress very slowly as we are making very tiny updates to the weights in our network.
- However, if learning rate is set too high, it can cause undesirable divergent behavior in our loss function.

Learning Rate Annealing



Learning Rate Annealing

Learning Rate Annealing

- One approach of using learning rate value properly is to start with large value of learning rate and reduce value of the learning rate according to some predefined schedule. This approach is called learning rate annealing.
- Three commonly used learning rate annealing techniques or learning rate schedules are:
 - Time Based Decay
 - Step Decay
 - Exponential Decay

Learning Rate Annealing

Time Based Decay

- Mathematical formulation of time-based learning rate decay is given below:

$$\alpha = \frac{\alpha_0}{1 + \text{decay_rate} \times \text{epoch\#}}$$

Example

Let $\alpha_0=0.2$ Decay_Rate=1

For Epoch=1 $\alpha=0.2/2=0.1$

For Epoch=2 $\alpha=0.2/3=0.067$

For Epoch=3 $\alpha=0.2/4=0.05$

So on

Learning Rate Annealing

Step Decay

- Mathematical formulation of step decay of learning rate is given below. A typical way is to drop the learning rate by half after every 10 epochs.

$$\alpha = \alpha_0 \times \text{drop_rate}^{(\lfloor \text{epoch\#} / \text{epochs_drop} \rfloor)}$$

Example

Let $\alpha_0=0.2$ $\text{drop_rate}=0.5$ $\text{epochs_drop}=5$

For Epoch=5 $\alpha=0.2 \times 0.5^{5/5}=0.1$

For Epoch=10 $\alpha=0.2 \times 0.5^{10/5}=0.05$

For Epoch=15 $\alpha=0.2 \times 0.5^{15/5}=0.025$

So on

Learning Rate Annealing

Exponential Decay

- Mathematical formulation of exponential decay of learning rate is given below.

$$\alpha = \alpha_0 e^{(-k \times epoch\#)}$$

Example

Let $\alpha_0=0.2$ $k=0.1$

For Epoch=1 $\alpha=0.181$

For Epoch=2 $\alpha=0.164$

For Epoch=3 $\alpha=0.148$

So on

Adaptive Control of Learning Rate

- Learning rate annealing techniques reduces learning rate equally for all parameters and applies the same learning rate with all parameters.
- However, in many practical situations, we need to change value of different parameters by different extent.
- Thus, the better approach is to use the techniques that gives learning rate a chance to adapt.

Adaptive Control of Learning Rate

Adaptive Control of Learning Rate

- The main idea behind adaptive learning rate is to use larger learning rate for updating parameters that are modified with small scale in past and use smaller learning rate for updating parameters that are modified with large scale in the past.
- Four commonly used learning rate adaptation techniques are: Momentum, Adagrad, Adadelata, RMSProp, and Adam.

Adaptive Control of Learning Rate

Momentum

- Momentum uses the past gradients to smooth out the parameter update.
- It computes an exponentially weighted average of gradients and then uses this gradient to update the weights.

$$v_t = \beta v_{t-1} + (1 - \beta) dw_t$$

$$w_{t+1} = w_t - \alpha v_t$$

Adaptive Control of Learning Rate

Adagrad

- In Adagrad, the variable v , called cache, keeps track of sum of the squared gradients, which in turn is used to normalize the parameter update.

$$v_t = v_{t-1} + dw_t^2$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} dw_t$$

Adaptive Control of Learning Rate

Adagrad

- The effect of above equations is that Weights receiving high gradients will have their effective learning rate reduced. On the other hand, weights receiving small gradients will have their effective learning rate increased.

Adaptive Control of Learning Rate

Adadelta

- Adadelta is an extension of Adagrad that seeks to reduce its aggressive, monotonically decreasing learning rate. Instead of accumulating all past squared gradients, Adadelta restricts the window of accumulated past gradients to some fixed size W .

Adaptive Control of Learning Rate

RMSProp

- Adagrad decays the learning rate very aggressively (as the denominator grows). As a result, after a while, parameters receiving larger gradients will start receiving very small updates because of the decayed learning rate.
- To avoid this why not decay the denominator and prevent its rapid growth.
- RMSProp stands for root mean squared propagation and avoids monotonically increasing denominator of Adagrad.

Adaptive Control of Learning Rate

RMSProp

- The central idea of RMSProp is keep the moving average of the squared gradients for each weight. And then divide the learning rate by root mean square of the squared gradients.

$$v_t = \beta v_{t-1} + (1 - \beta) dw_t^2$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} dw_t$$

Adaptive Control of Learning Rate

Adam

- Adam update can be considered as RMSprop with momentum. In place of raw and noisy gradient vector dw , weighted average of gradients are used.

Weighted average of gradients

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) dw_t$$

Weighted average of squared gradients

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) dw_t^2$$

Adaptive Control of Learning Rate

Adam

Bias Correction

$$\mathbf{v}_t = \frac{\mathbf{v}_t}{(1 - \beta_1^t)} \quad S_t = \frac{S_t}{(1 - \beta_2^t)}$$

Finally, update weights as below:

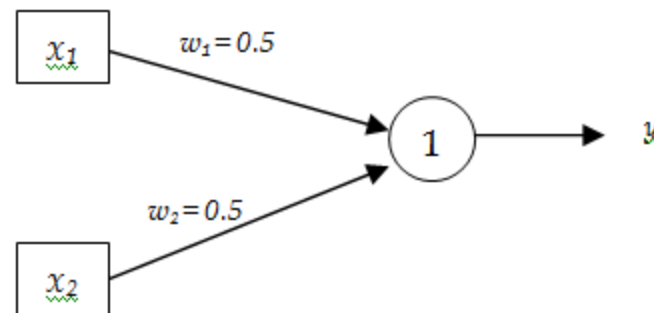
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{\sqrt{S_t} + \epsilon} \mathbf{v}_t$$

Adaptive Control of Learning Rate

Example

- Consider following simple ANN with logistic activation function. Assume that $\alpha = 0.2$ $\beta = 0.8$ $\beta_1 = 0.8$ $\beta_2 = 0.8$. Calculate weight updates for the given training Sample using (1) Momentum (2) Adagrad (3) RMSProp and (4) Adam

x_1	x_2	t
1	0.6	0.8
0.5	0.5	0.6
0.2	0.8	0.3



Adaptive Control of Learning Rate

Solution

For Adagrad

For Training example:
(1,0.6,0.8)

$$y_{in} = 0.5 * 1 + 0.5 * 0.6 = 0.8$$

$$y = \frac{1}{1 + \exp(-y_{in})} = 0.6899$$

$$dw_1 = (y - t) \times y \times (1 - y) \times x_1 = -0.02354$$

$$dw_2 = (y - t) \times y \times (1 - y) \times x_2 = -0.01412$$

$$v_1 = -0.02354^2 = 0.000554$$

$$v_2 = -0.01412^2 = 0.000199$$

$$w_1 = w_1 - \frac{0.2}{\sqrt{0.000554}} \times (-0.02354) = 0.7$$

$$w_2 = w_2 - \frac{0.2}{\sqrt{0.000199}} \times (-0.01412) = 0.7$$

Adaptive Control of Learning Rate

Solution

For Adagrad

For Training example:
(0.5,0.5,0.6)

$$y_{in} = 0.7 * 0.5 + 0.7 * 0.5 = 0.7$$

$$y = \frac{1}{1 + \exp(-y_{in})} = 0.6682$$

$$dw_1 = (y - t) \times y \times (1 - y) \times x_1 = 0.007559$$

$$dw_2 = (y - t) \times y \times (1 - y) \times x_2 = 0.007559$$

$$v_1 = 0.000554 + 0.007559^2 = 0.000611$$

$$v_2 = 0.000199 + 0.007559^2 = 0.000257$$

$$w_1 = w_1 - \frac{0.2}{\sqrt{0.000611}} \times 0.007559 = 0.639$$

$$w_2 = w_2 - \frac{0.2}{\sqrt{0.000257}} \times 0.007559 = 0.606$$

Adaptive Control of Learning Rate

Solution

For Adagrad

For Training example:
(0.2,0.8,0.3)

$$y_{in} = ????$$

$$y = \frac{1}{1 + \exp(-y_{in})} = ??$$

$$dw_1 = (y - t) \times y \times (1 - y) \times x_1 = ??$$

$$dw_2 = (y - t) \times y \times (1 - y) \times x_2 = ??$$

$$v_1 = ??$$

$$v_2 = ??$$

$$w_1 = ??? = 0.531$$

$$w_2 = ??? = 0.412$$

Adaptive Control of Learning Rate

Solution

For RMSProp

For Training

example:(1,0.6,0.8)

$$y_{in} = 0.5 * 1 + 0.5 * 0.6 = 0.8$$

$$y = \frac{1}{1 + \exp(-y_{in})} = 0.6899$$

$$dw_1 = (y - t) \times y \times (1 - y) \times x_1 = -0.02354$$

$$dw_2 = (y - t) \times y \times (1 - y) \times x_2 = -0.01412$$

$$v_1 = 0.8 * 0 + (1 - 0.8) * (-0.02354)^2 = 0.000111$$

$$v_2 = 0.8 * 0 + (1 - 0.8) * (-0.01412)^2 = 0.0000399$$

$$w_1 = w_1 - \frac{0.2}{\sqrt{0.000111}} \times (-0.02354) = 0.947$$

$$w_2 = w_2 - \frac{0.2}{\sqrt{0.0000399}} \times (-0.01412) = 4.971$$

Adaptive Control of Learning Rate

Solution

For RMSProp

For Training

example:(0.5,0.5,0.6)

$$y_{in} = 0.947 * 0.5 + 4.971 * 0.5 = ??$$

$$y = \frac{1}{1 + \exp(-y_{in})} = ??$$

$$dw_1 = (y - t) \times y \times (1 - y) \times x_1 = ???$$

$$dw_2 = (y - t) \times y \times (1 - y) \times x_2 = ???$$

$$v_1 = 0.8 * 0.000111 + (1 - 0.8) * (dw_1)^2 = ????$$

$$v_2 = 0.8 * 0.0000399 + (1 - 0.8) * (dw_2)^2 = ???$$

$$w_1 = w_1 - \frac{0.2}{\sqrt{v_1}} \times (dw_1) = ???$$

$$w_2 = w_2 - \frac{0.2}{\sqrt{v_2}} \times (dw_2) = ???$$

Adaptive Control of Learning Rate

Solution

For RMSProp

For Training

example:(0.2,0.8,0.3)

$$y_{in} = ???$$

$$y = \frac{1}{1 + \exp(-y_{in})} = ??$$

$$dw_1 = (y - t) \times y \times (1 - y) \times x_1 = ???$$

$$dw_2 = (y - t) \times y \times (1 - y) \times x_2 = ???$$

$$v_1 = 0.8 * ??? + (1 - 0.8) * (dw_1)^2 = ????$$

$$v_2 = 0.8 * ??? + (1 - 0.8) * (dw_2)^2 = ???$$

$$w_1 = w_1 - \frac{0.2}{\sqrt{v_1}} \times (dw_1) = ???$$

$$w_2 = w_2 - \frac{0.2}{\sqrt{v_2}} \times (dw_2) = ???$$

Adaptive Control of Learning Rate

Solution

For Momentum and Adam

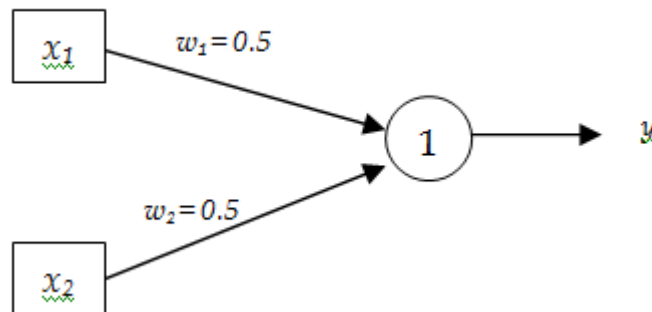
??????---try yourself

Adaptive Control of Learning Rate

Example

- Consider following simple ANN with logistic activation function. Assume that $\alpha = 0.2$ $\beta = 0.8$ $\beta_1 = 0.8$ $\beta_2 = 0.8$. Calculate weight updates for the given training Sample using (1) Momentum (2) Adagrad (3) RMSProp and (4) Adam.
- Note: Use bias correction feature in each case

x_1	x_2	t
1	0.6	0.8
0.5	0.5	0.6
0.2	0.8	0.3



Cross-Validation

- In machine learning, datasets are divided into three sets: Training Set, Validation Set, and Test Set. Split ratio of dataset into three sets is not fix. It depends upon size of the dataset. If we have very large dataset split ratio can be 98:1:1. Otherwise, we can opt for split ratio 80:10:10 or 70:15:15 so on.
- Training set is used to train neural network and update weights on the basis of error. Validation set is used to compute prediction error using weights resulted from training. If the error is higher than a user-defined threshold then the whole training-validation epoch is repeated.

Cross-Validation

- The testing set is then used to measure the performance of the network. This is the data that was never used throughout the training and validation phase.
- *Cross-validation is a re-sampling procedure used to evaluate machine learning models on a limited data sample.*
- k-Fold Cross-Validation (Muti-Fold Cross-Validation) is widely used for cross validation. If we choose $k=10$, it becomes 10-fold cross validation.

Cross-Validation

- In this approach, the original sample is randomly partitioned into k equal sized subsamples. Out of the k subsamples, a single subsample is used as the testing set, and the remaining $k - 1$ subsamples are used as training+validation data.
- The cross-validation process is then repeated k times, with each of the k subsamples used exactly once as the testing data. The k results can then be averaged to produce a single estimation.

Cross-Validation

- The advantage of this method is that all observations are used for both training and testing, and each observation is used for testing exactly once. 10-fold cross-validation is commonly used, but in general k remains an unfixed parameter.
- For example, setting $k = 2$ results in 2-fold cross-validation. In 2-fold cross-validation, we randomly shuffle the dataset into two sets d_0 and d_1 , so that both sets are equal size. We then train on d_0 and test on d_1 , followed by training on d_1 and testing on d_0 .

Cross-Validation

- When $k = n$ (the number of observations), k -fold cross-validation is equivalent to leave-one-out cross-validation. In this case, $n - 1$ examples are used to train the model, and the model is validated by testing it on the example that is left out. This approach is suitable when we have small number of training example.
- To make the cross-validation procedure concrete, let's look at a worked example. Imagine we have a data sample with 6 observations:
 $\{(1,3),(2,5),(3,7),(4,9),(5,11),(6,13)\}$
- Let $k=3$. That means we will shuffle the data and then split the data into 3 groups.

Cross-Validation

- Let $k=3$. That means we will shuffle the data and then split the data into 3 groups.

Fold 1 = $\{(1,3), (5,11)\}$

Fold 2 = $\{(6,13), (3,7)\}$

Fold 3 = $\{(2,5), (4,9)\}$

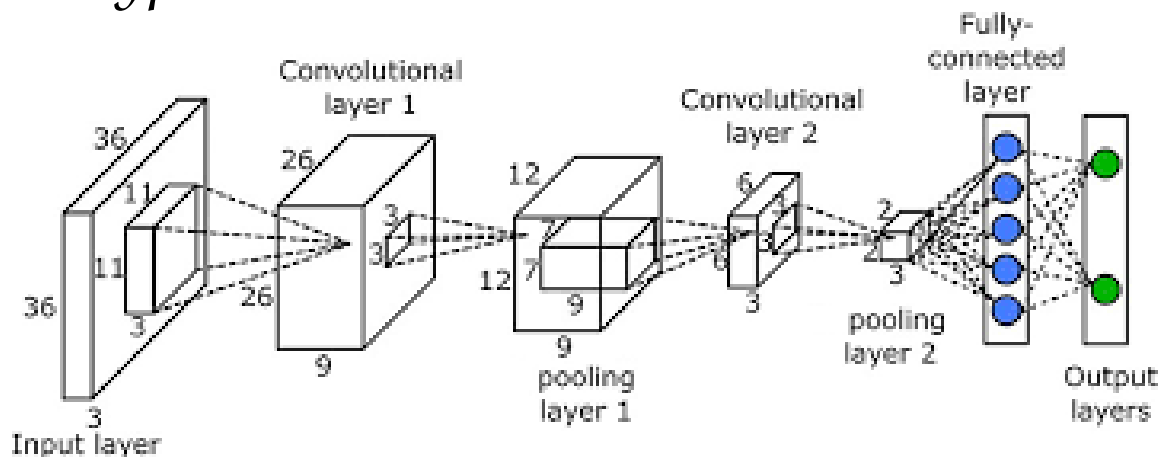
- Three models are trained and evaluated with each fold given a chance to be the held out test set. For example:
 - **Model 1:** Trained on Fold 1 + Fold 2, Tested on Fold 3
 - **Model 2:** Trained on Fold 2 + Fold 3, Tested on Fold 1
 - **Model 3:** Trained on Fold 1 + Fold 3, Tested on Fold 2

Cross-Validation

- In typical cross-validation, results of multiple runs of model-testing are averaged together. In contrast, the holdout method, involves a single run.
- It should be used with caution because without such averaging of multiple runs, one may achieve highly misleading results. Predictive accuracy will tend to be unstable since it will not be smoothed out by multiple iterations.

Convolutional Neural Network(CNN)

- CNN is a type of multilayer neural network specially designed for detecting, recognizing, and classifying images.
- Every CNN consist following three types of layers: *Convolution Layer, Pooling layer, and Fully-connected Layer*. Typical architecture of CNN is shown below:



Convolutional Neural Network(CNN)

Convolution Layer

- Convolution layer extracts features from an input image. It preserves the relationship between pixels by learning image features using small squares of input data. It is a mathematical operation that takes two inputs such as image matrix and a filter or kernel and produces an image with convolved features.
- Consider a 5×5 image whose pixel values are 0, 1 and filter matrix 3×3 as shown in below.
- The matrix formed by sliding the filter over the image and computing the dot product is called Feature Map or Convolved Feature.

Convolutional Neural Network(CNN)

Convolution Layer

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

5 x 5 – Image Matrix

1	0	1
0	1	0
1	0	1

3 x 3 – Filter Matrix



4	3	4
2	4	3
2	3	4

Feature Map

- Convolution of an image with different filters can perform operations such as edge detection, blur and sharpen by applying filters.

Convolutional Neural Network(CNN)

Pooling Layer

- Spatial Pooling or simply Pooling reduces the dimensionality of each feature map but retains the most important information.
- It is also called sub-sampling or down-sampling. Spatial Pooling can be of different types: *Max, Average, Sum etc.*
- In case of Max Pooling, we define a spatial neighborhood (for example, a 3×3 window) and take the largest element from the rectified feature map within that window.

Convolutional Neural Network(CNN)

Pooling Layer

- Instead of taking the largest element we could also take the average (Average Pooling) or sum of all elements in that window. In practice, Max Pooling has been shown to work better.

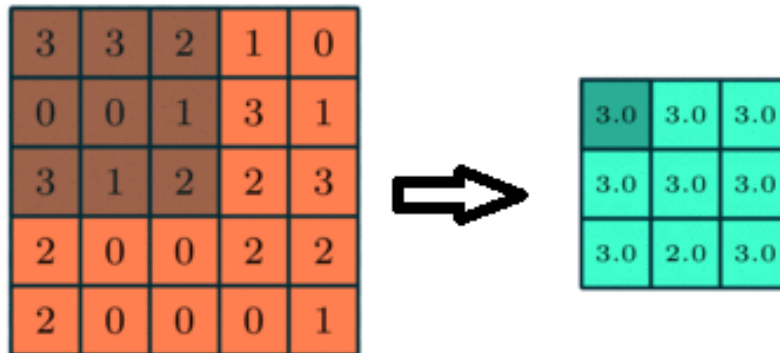


Figure: 3x3 Max Pooling over 5x5 convolved feature

Convolutional Neural Network(CNN)

Fully Connected Layer

- The Fully Connected layer is a traditional Multi Layer Perceptron that uses a softmax activation function in the output layer. The term “*Fully Connected*” implies that every neuron in the previous layer is connected to every neuron on the next layer.
- The output from the Convolutional and Pooling layers represent high-level features of the input image. The purpose of the Fully Connected layer is to use these features for classifying the input image into various classes based on the training dataset.

Convolutional Neural Network(CNN)

Fully Connected Layer

- The Softmax function takes a vector of arbitrary real-valued scores and squashes it to a vector of values between zero and one that sum to one.
- The Fully-Connected layer learns non-linear function in output produced by Convolutional and Pooling Layer.
- We should flatten the image into a column vector. The flattened output is fed to the Fully Connected Layer and backpropagation can be applied to train the network.

Convolutional Neural Network(CNN)

Example

Consider following 6x6 image and 3x3 filter as below. Compute feature map and pooled feature map using 2x2 window. Use Max pooling.

255	200	120	89	0	180
210	230	170	165	87	76
49	120	115	125	165	140
20	35	32	42	37	78
55	65	75	45	35	69
190	180	160	150	155	165

1	1	0
0	1	0
0	1	1

Fast Learning Method: Newton's Method

- One of the main problem with gradient descent is slow convergence.
- Newton's method is a second-order algorithm because it makes use of the Hessian matrix.
- This method's objective is to find better training directions by using the second derivatives of the loss function.
- Second order Taylor series expansion of the cost function around point $w(n)$ is given as below.

Fast Learning Method: Newton's Method

$$\begin{aligned}\Delta F(w(n)) &= F(w(n+1)) - F(w(n)) \\ &= g^T(n)w(n) + \frac{1}{2}\Delta w^T(n)H(n)w(n)\end{aligned}\quad (1)$$

- Where, g is gradient vector of cost function evaluated at $w(n)$ and $H(n)$ is hessian evaluated at $w(n)$.
- Differentiating eq(1) w.r.t. $w(n)$ and equating it with zero, we get.

$$\begin{aligned}g(n) + w(n)H(n) &= 0 \\ w(n) &= -g(n)H^{-1}(n)\end{aligned}\quad (2)$$

- Since, $\Delta w(n) = w(n+1) - w(n)$

Fast Learning Method: Newton's Method

$$\begin{aligned}\Rightarrow \quad w(n+1) &= w(n) + \Delta w(n) \\ &= w(n) - g(n)H^{-1}(n)\end{aligned}\tag{3}$$

- This equation is used to update weights of neural network.
- Newton's method converges quickly and does not exhibit the zigzagging behavior that sometimes characterizes the method of gradient descent.
- However, for Newton's method to work, the Hessian $\mathbf{H}(n)$ has to be a positive definite matrix for all n .

Fast Learning Method: Newton's Method

- Unfortunately, in general, there is no guarantee that $\mathbf{H}(n)$ is positive definite at every iteration of the algorithm.
- In any event, a major limitation of Newton's method is its computational complexity.

Fast Learning Method: Conjugate Gradient

- The Backpropagation algorithm adjusts the weights in the steepest descent direction (negative of the gradient).
- This is the direction in which the cost function decreases most rapidly.
- Although the function decreases most rapidly along the negative of the gradient, this does not necessarily produce the fastest convergence due to zigzag nature of convergence path.

Fast Learning Method: Conjugate Gradient

- If gradient descent finds minimum in each search direction, successive search directions will be orthogonal to each other. This is the reason behind zig-zag convergence path of the gradient descent.

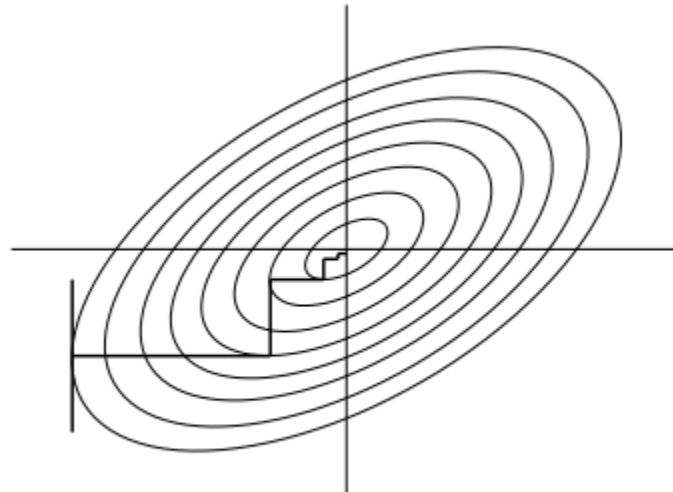


Fig. Convergence of Gradient Descent

Fast Learning Method: Conjugate Gradient

- In Backpropagation, a learning rate is used to determine the length of the weight update (step size).
- However, In the conjugate gradient algorithms, the step size is adjusted at each iteration. A search is made along the conjugate gradient direction to determine the step size, which minimizes the cost function along that line.

Fast Learning Method: Conjugate Gradient

- The concept behind conjugate gradient descent is that since each step is taking the same general direction, the combination of search directions should minimize the zig-zag nature of convergence path and capture the general trend towards the minimum.

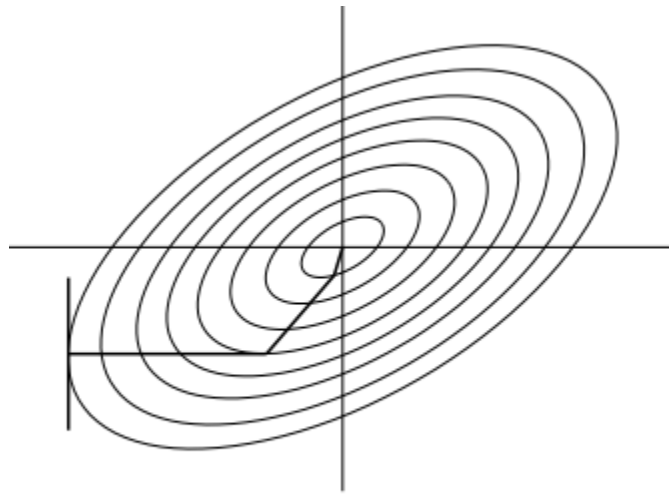


Fig. Convergence of Conjugate Gradient

Fast Learning Method: Conjugate Gradient

- All of the conjugate gradient algorithms start out by searching in the steepest descent direction (negative of the gradient) on the first iteration.

$$d_0 = -g_0$$

- Then the next search direction is determined so that it is conjugate to previous search directions. The general procedure for determining the new search direction is to combine the new steepest descent direction with the previous search direction.

$$d_k = -g_k + \beta_k d_{k-1}$$

Fast Learning Method: Conjugate Gradient

- Here β is called the conjugate parameter, and there are different ways to calculate it. Using the Fletcher-Reeves procedure β can be calculated as below.

$$\beta_k = \frac{g_k^T g_k}{g_{k-1}^T g_{k-1}}$$

This is the ratio of the squared norm of the current gradient to the squared norm of the previous gradient.

Fast Learning Method: Conjugate Gradient

- Finally, parameters are updated to move along the current search direction as below.

$$w_{k+1} = w_k + \alpha_k d_k$$

- It is an effective search method for minimizing objective functions if the number of parameters to be optimized are few.

Auto associative Neural Network

- This is a fully-connected neural network in which the input training vector and the output target vectors are the same.
- The weights are determined so that the network stores a set of patterns.
- A stored vector can be retrieved from a distorted or noisy vector if the input is similar to it.
- This network can be trained using Hebbian learning rule or Heb's rule.

Auto associative Neural Network

- The rule is based on the assumption that if two neighbor neurons activated and deactivated at the same time. Then the weight connecting these neurons should increase. For neurons operating in the opposite phase, the weight between them should decrease. If there is no signal correlation, the weight should not change.
- This assumption can be described by using following formula.

Auto associative Neural Network

$$\Delta w_{ij} = \alpha x_i y_j$$

where, w_{ij} is weight from neuron i to neuron j

α is learning rate

x_i is input

y_j is output

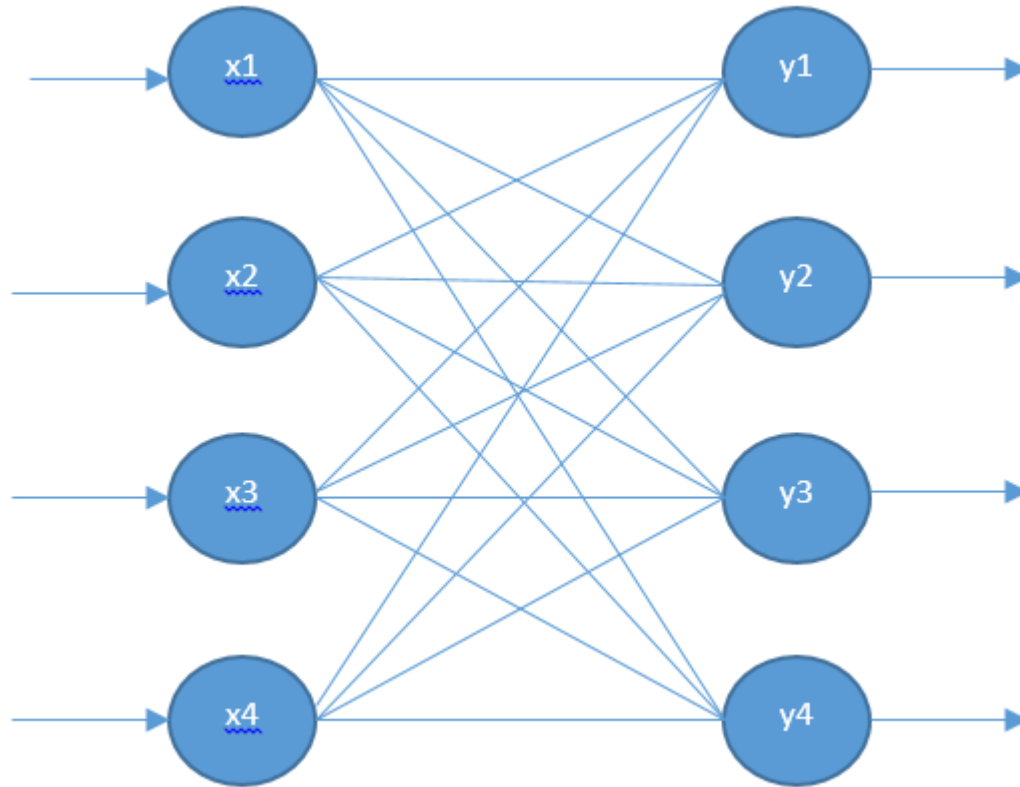
- Activation function for auto associative neural network is:

$$y_j = 1 \text{ if } y_{in} > c,$$

$$y_j = x_j \text{ if } y_{in} = c \text{ and}$$

$$y_j = -1 \text{ if } y_{in} < c$$

Auto associative Neural Network



Architecture of AANN

Auto associative Neural Network

Training Algorithm

1. Initialize all weights and bias to zero
2. For each vector to be stored (Say s) perform steps 3 to 5
3. For each input unit i , set $x_i = s_i$
4. For each output unit j set $y_j = s_j$
5. Adapt weights as: $w_{ij} = w_{ij} + \alpha x_i y_j$

Weights can also be determined as: $w = s^T \cdot s$

Auto associative Neural Network

Example

Use the Heb's rule to store the pattern $[1, 1, -1, -1]$ in an AANN. Find weight matrix and Test the input vector $[-1, 1, -1, -1]$.

Solution

Here, pattern to be stored(s) = $[1, 1, -1, -1]$

$$\text{Now, } w = \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} [1 \quad 1 \quad -1 \quad -1] = \begin{bmatrix} 1 & 1 & -1 & -1 \\ 1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \end{bmatrix}$$

Auto associative Neural Network

Now,

Test input= $[-1 \quad 1 \quad -1 \quad -1]$

The calculate $y_{in} = x.w = [2 \quad 2 \quad -2 \quad -2]$

Finally compute $y = [1 \quad 1 \quad -1 \quad -1]$

Bayesian Neural Network

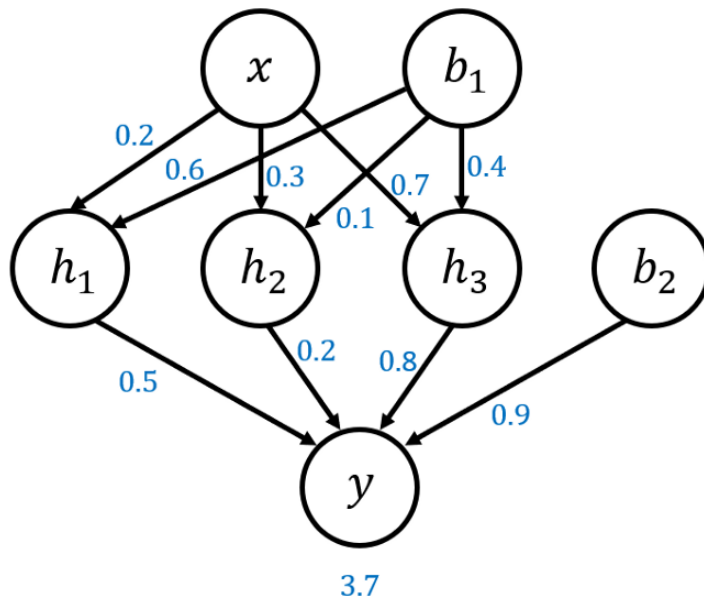
- Taking a probabilistic approach to neural network allows to account for *uncertainty*, so that models can assign less levels of confidence to incorrect predictions.
- Sources of uncertainty can be found in the data, due to measurement error or noise in the labels, or the model, due to insufficient data availability for the model to learn effectively.

Bayesian Neural Network

- The idea behind Bayesian neural network is that, instead of learning specific weight (and bias) *values* in the neural network, the Bayesian approach learns weight *distributions* - from which we can sample to produce an output for a given input - to encode weight uncertainty.

Bayesian Neural Network

Standard Neural Network



Bayesian Neural Network

